AI_PROJECT

1. Traffic Light Optimizer >>

Problem Description

Simulate an intersection with 4 traffic lights that adaptively change signals to reduce vehicle waiting time and improve throughput.

Solution Concept

Each traffic light is an **agent** that monitors:

- Queue length
- Green timer
- Neighbor signal queues

They decide when to switch from RED ↔ GREEN based on local traffic conditions.

Algorithm Used

Rule-Based Adaptive Control

- If queue too long → switch to GREEN
- If GREEN duration exceeded max limit → switch to RED
- Otherwise continue current state

AI / Agent Concepts

- **Multi-agent coordination**: Each signal is an independent agent
- **Perception**: Reads queue length
- **Action**: Switch signal state
- **Learning**: Simple heuristic (can be extended to RL)
- **Environment**: Dynamic, partially observable

PEAS Model

- **Performance**: Avg waiting time, throughput
- **Environment**: Vehicles, intersection
- **Actuators**: Signal lights
- **Sensors**: Queue counters

CODE:

```
# Traffic Light Optimizer — graphical simulation (matplotlib animation)
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np, random
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(1); np.random.seed(1)

# Vehicle and Signal classes
class Vehicle:
    def __init__(self, vid, direction):
        self.id = vid
        self.direction = direction
        self.color = random.choice(['red','blue','green','orange','purple','brown'])
        self.pos = self.start_pos()
        self.waiting = 0
        self.speed = 6
        self.exited = False
    def start_pos(self):
        if self.direction=='N': return [500, 900]
        if self.direction=='S': return [520, -50]
        if self.direction=='E': return [-50, 520]
        return [1050, 500]
    def move(self, green):
        if green:
```

```
            if self.direction=='N': self.pos[1] -= self.speed
            if self.direction=='S': self.pos[1] += self.speed
            if self.direction=='E': self.pos[0] += self.speed
            if self.direction=='W': self.pos[0] -= self.speed
        else:
            self.waiting += 1
        # exit bounds
        if self.pos[0] < -100 or self.pos[0] > 1100 or self.pos[1] < -150 or self.pos[1] > 1050:
            self.exited = True

class Signal:
    def __init__(self, direction, pos):
        self.direction = direction; self.pos = pos
        self.state = 'RED'; self.green_timer = 0
        self.min_green = 20; self.max_green = 70
        self.queue = []
    def qlen(self): return len(self.queue)
    def decide(self, others):
        # If green too long -> switch
        if self.state=='GREEN' and self.green_timer >= self.max_green:
            return 'SWITCH'
        # If red and this queue longer than others -> switch
        if self.state=='RED' and self.qlen() > max((o.qlen() for o in others), default=0):
            return 'SWITCH'
        return 'CONTINUE'
    def update(self, others):
        action = self.decide(others)
        if action=='SWITCH':
            self.state = 'GREEN' if self.state=='RED' else 'RED'
            self.green_timer = 0
        else:
            self.green_timer += 1
        # remove vehicles that have passed intersection (approx)
        self.queue = [v for v in self.queue if not self.passed(v)]
```

```
    def passed(self, v):
        if self.direction=='N': return v.pos[1] < 440
        if self.direction=='S': return v.pos[1] > 560
        if self.direction=='E': return v.pos[0] > 560
        if self.direction=='W': return v.pos[0] < 440
        return False

# Simulation env
class TrafficSim:
    def __init__(self):
        self.signals = [
            Signal('N', (500,450)),
            Signal('S', (520,550)),
            Signal('E', (550,500)),
            Signal('W', (450,520))
        ]
        self.vehicles = []
        self.vid = 0
        self.time = 0
        self.spawn_rates = {'N':0.07,'S':0.07,'E':0.07,'W':0.07}
        self.throughput = 0
    def spawn(self):
        for s in self.signals:
            if random.random() < self.spawn_rates[s.direction]:
                self.vid += 1
                v = Vehicle(self.vid, s.direction)
                self.vehicles.append(v)
                s.queue.append(v)
    def step(self):
        self.time += 1
        self.spawn()
        for s in self.signals:
            others = [o for o in self.signals if o!=s]
            s.update(others)
```

```
        for v in list(self.vehicles):
            sig_state = next(s.state for s in self.signals if s.direction==v.direction)
            v.move(sig_state=='GREEN')
            if v.exited:
                self.vehicles.remove(v)
                self.throughput += 1

sim = TrafficSim()
fig, ax = plt.subplots(figsize=(10,8))

def animate(i):
    ax.clear()
    sim.step()
    # draw roads
    ax.add_patch(plt.Rectangle((0,480),1000,40,color='dimgray',zorder=0))
    ax.add_patch(plt.Rectangle((480,0),40,900,color='dimgray',zorder=0))
    # draw vehicles
    for v in sim.vehicles:
        if v.direction in ['N','S']:
            ax.add_patch(plt.Rectangle((v.pos[0]-7, v.pos[1]-15), 15, 30, color=v.color))
        else:
            ax.add_patch(plt.Rectangle((v.pos[0]-15, v.pos[1]-7), 30, 15, color=v.color))
    # draw signals
    for s in sim.signals:
        col = 'green' if s.state=='GREEN' else 'red'
        ax.plot(s.pos[0], s.pos[1], 'o', color=col, markersize=18, markeredgecolor='black')
        ax.text(s.pos[0], s.pos[1]+35, f"{s.direction} Q:{s.qlen()} T:{s.green_timer}", ha='center')
    # metrics
    avg_wait = np.mean([v.waiting for v in sim.vehicles]) if sim.vehicles else 0
    ax.text(20, 20, f"Time:{sim.time} Vehicles:{len(sim.vehicles)} AvgWait:{avg_wait:.1f}
Throughput:{sim.throughput}", fontsize=10)
    ax.set_xlim(0,1000); ax.set_ylim(0,900); ax.axis('off'); ax.set_title("Traffic Light Optimizer")
ani = animation.FuncAnimation(fig, animate, frames=400, interval=120)
plt.close(); HTML(ani.to_jshtml())
```

==========

2. Garbage Collection Routing>> truck route on grid, animated traversal:

Problem Description

Garbage trucks must visit multiple pickup points using minimal distance/time.

Solution Concept

A truck agent travels a city grid and selects the **nearest unvisited garbage point** until all are collected.

Algorithm Used

Greedy Nearest Neighbor + Dijkstra Shortest Path

- Build path incrementally
- Use Dijkstra for shortest between points

AI Concepts

- **Path planning**
- **Graph search**
- **Optimization** problem

PEAS

- **Performance**: Total distance, time
- **Environment**: Road network graph
- **Actuator**: Truck movement
- **Sensor**: Graph distances

CODE:

```
# Garbage Collection Routing — animated traversal on grid (matplotlib)
import matplotlib.pyplot as plt, matplotlib.animation as animation
import networkx as nx, random
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(2)

# build grid graph
G = nx.grid_2d_graph(6,6)
G = nx.convert_node_labels_to_integers(G)
pos = {n: (n%6, n//6) for n in G.nodes()}
stops = random.sample(list(G.nodes()), 8)
depot = 0

# greedy route builder (nearest neighbor)
def route_from(start, stops):
    cur = start; route = [cur]; un = set(stops)
    while un:
        nxt = min(un, key=lambda x: nx.shortest_path_length(G, cur, x))
        path = nx.shortest_path(G, cur, nxt)
        route += path[1:]; cur = nxt; un.remove(nxt)
    route += nx.shortest_path(G, cur, start)[1:]
    return route

route = route_from(depot, stops)
# expand route into sequence of coordinates
coords = [pos[n] for n in route]

fig, ax = plt.subplots(figsize=(6,6))
truck_idx = 0

def animate(i):
    global truck_idx
    ax.clear()
    nx.draw(G, pos=pos, node_color='lightgrey', with_labels=False, node_size=200, ax=ax)
    nx.draw_networkx_nodes(G, pos, nodelist=stops, node_color='red', node_size=300,
ax=ax)
    ax.scatter(*pos[depot], c='green', s=300)
    # draw route lines
    for a,b in zip(route[:-1], route[1:]):
        x1,y1 = pos[a]; x2,y2 = pos[b]
        ax.plot([x1,x2],[y1,y2], color='lightblue', linewidth=1)
    # animate truck along coords
    idx = min(i, len(coords)-1)
    tx,ty = coords[idx]
    ax.scatter(tx,ty, s=200, marker='s', c='orange')
    ax.set_title(f"Garbage Truck Route — step {idx+1}/{len(coords)}")
    ax.set_xlim(-0.5,5.5); ax.set_ylim(-0.5,5.5); ax.axis('off')
```

ani = animation.FuncAnimation(fig, animate, frames=len(coords)+10, interval=350, repeat=False)
plt.close(); HTML(ani.to_jshtml())

==========

3) Smart Parking Allocation — animated parking lot filling (heatmap + vehicles)>>

Problem Description

Assign vehicles to nearest available parking spots dynamically.

Solution Concept

Each incoming vehicle acts as an agent looking for the closest free spot. A central allocator assigns spots greedily.

Algorithm Used

Greedy Assignment Algorithm
(Often Hungarian algorithm for optimal matching, but greedy is faster)

AI Concepts

- **Resource allocation**
- **Constraint satisfaction**
- **Grid-based perception**

PEAS

- **Performance**: Walking distance, occupancy rate
- **Environment**: Parking lot grid
- **Actuator**: Assigning spots
- **Sensor**: Spot availability

CODE:

Smart Parking Allocation — animated lot filling (matplotlib)

```
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np, random
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(3); np.random.seed(3)

rows, cols = 10, 6
spots = [(r,c) for r in range(rows) for c in range(cols)]
free = set(spots)
vehicles = []
assignments = {}
steps = 120
spawn_prob = 0.25

fig, ax = plt.subplots(figsize=(6,5))

def find_nearest(v):
    if not free: return None
    return min(free, key=lambda s: (s[0]-v[0])**2 + (s[1]-v[1])**2)

def animate(t):
    ax.clear()
    # new arrivals
    if random.random() < spawn_prob and len(vehicles) < 60:
        v = (random.uniform(0,rows-1), random.uniform(0,cols-1))
        vehicles.append({'pos':v, 'assigned':None})
    # assign greedy
    for v in vehicles:
        if v['assigned'] is None:
            nearest = find_nearest(v['pos'])
            if nearest is not None:
                v['assigned'] = nearest
                free.remove(nearest)
                assignments[tuple(v['pos'])] = nearest
    # draw parking grid heatmap of occupancy
    heat = np.zeros((rows,cols))
    for a in assignments.values(): heat[a] += 1
    ax.imshow(heat.T, origin='lower', extent=[-0.5, rows-0.5, -0.5, cols-0.5])
    # draw vehicles as dots moving towards assigned spot
    for v in vehicles:
        px,py = v['pos']
        if v['assigned']:
            sx,sy = v['assigned']
            # move a bit toward
            nxp = px + (sx - px) * 0.25
            nyp = py + (sy - py) * 0.25
            v['pos'] = (nxp, nyp)
            ax.plot([px, nxp], [py, nyp], color='gray', linewidth=1)
```

```
        ax.scatter(v['pos'][0], v['pos'][1], c='black', s=20)
    ax.set_title(f"Smart Parking Allocation — occupied: {len(assignments)} free: {len(free)}")
    ax.set_xlim(-0.5, rows-0.5); ax.set_ylim(-0.5, cols-0.5); ax.set_xticks([]); ax.set_yticks([])

ani = animation.FuncAnimation(fig, animate, frames=steps, interval=200)
plt.close(); HTML(ani.to_jshtml())
```

==========

4) Energy Distribution Agents — buildings demand vs allocation animated bar chart>>

Problem Description

Distribute limited electricity among several buildings fairly based on demand.

Solution Concept

Compute proportional allocation of total power to each building according to its demand.

Algorithm Used

Proportional Allocation Algorithm

- alloc[i] = demand[i] / total_demand * supply

AI Concepts

- **Distributed resource management**
- **Agent negotiation**
- **Load balancing**

PEAS

- **Performance**: Load variance, % unmet demand
- **Environment**: Microgrid
- **Actuator**: Distribution controller
- **Sensor**: Demand readings

CODE:

```
# Energy Distribution Agents — animated demand vs allocated bars
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
np.random.seed(4)

n_buildings = 8
T = 120
# generate time-varying demands
demand = (np.random.poisson(8, (T, n_buildings)) +
np.round(6*np.sin(np.linspace(0,6,T))[:,None])).clip(min=1)
supply = (50 + 12*np.sin(np.linspace(0,3.5,T))).astype(int)

fig, ax = plt.subplots(figsize=(8,4))

def allocate(t):
    d = demand[t].astype(float)
    total_d = d.sum()
    s = supply[t]
    if total_d <= s:
        return d
    else:
        alloc = np.floor(d / total_d * s)
        # adjust remainder
        rem = int(s - alloc.sum())
        if rem>0:
            for i in np.argsort(-d)[:rem]:
                alloc[i] += 1
        return alloc

def animate(i):
    ax.clear()
    alloc = allocate(i)
    ax.bar(range(n_buildings), demand[i], alpha=0.5, label='Demand')
    ax.bar(range(n_buildings), alloc, alpha=0.9, label='Allocated')
    ax.set_ylim(0, max(demand.max(), supply.max())+10)
    ax.set_title(f"Energy Distribution t={i} supply={supply[i]}")
    ax.legend()
    ax.set_xlabel('Building')
    ax.set_ylabel('Energy units')

ani = animation.FuncAnimation(fig, animate, frames=T, interval=150)
plt.close(); HTML(ani.to_jshtml())
```

==========

5) Water Supply Optimizer — animated proportional allocation to houses

Problem Description

Distribute limited water supply across houses ensuring fairness.

Solution Concept

Calculate proportional distribution based on demand while respecting total supply.

Algorithm Used

Constraint-Based Proportional Distribution
Used in utility allocation problems.

AI Concepts

- **Constraint satisfaction**
- **Optimization**
- **Fair division algorithms**

PEAS

- **Performance**: Demand-satisfaction ratio
- **Environment**: Water network
- **Actuator**: Valve control
- **Sensor**: House demand

CODE:

```
>> # Water Supply Optimizer — animated proportional allocation
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
```

```
np.random.seed(5)

houses = 12
T = 100
demand = np.random.randint(5,30,(T,houses))
supply = (120 + 20*np.sin(np.linspace(0,4,T))).astype(int)

fig, ax = plt.subplots(figsize=(8,4))
def allocate(t):
    d = demand[t].astype(float)
    s = supply[t]
    total = d.sum()
    if total <= s:
        return d
    else:
        alloc = np.floor(d / total * s)
        # fix rounding
        rem = int(s - alloc.sum())
        for idx in np.argsort(-d)[:rem]:
            alloc[idx] += 1
        return alloc

def animate(i):
    ax.clear()
    alloc = allocate(i)
    ax.bar(np.arange(houses)-0.2, demand[i], width=0.4, label='Demand', alpha=0.6)
    ax.bar(np.arange(houses)+0.2, alloc, width=0.4, label='Allocated', alpha=0.9)
    ax.set_title(f"Water Distribution t={i} supply={supply[i]}")
    ax.set_xlabel("House"); ax.set_ylabel("Flow units"); ax.legend()

ani = animation.FuncAnimation(fig, animate, frames=T, interval=150)
```

plt.close(); HTML(ani.to_jshtml())

==========

6) Emergency Response Dispatchers — animated incidents and nearest assignment>>

Problem Description

Assign ambulances/fire trucks to incidents to minimize response time.

Solution Concept

For each new incident, assign the closest available emergency station.

Algorithm Used

Greedy Nearest Neighbor Assignment

- Compute Euclidean distance for each station
- Pick minimum

AI Concepts

- **Reactive agents**
- **Decision-making under time pressure**
- **Geospatial reasoning**

PEAS

- **Performance**: Avg response time
- **Environment**: City map
- **Actuator**: Dispatch vehicles
- **Sensor**: Incident coordinates

CODE:

```
# Emergency Response Dispatchers — animated incidents assigned to stations
import matplotlib.pyplot as plt, matplotlib.animation as animation
import random, math
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(6)

stations = [(10,10),(90,10),(50,90)]
# dynamic incidents spawn over time
incidents = []
assignments = []  # (incident, station, spawn_time)
T = 80

fig, ax = plt.subplots(figsize=(6,6))
def animate(t):
    ax.clear()
    # spawn incident with small prob
    if random.random() < 0.28:
        inc = (random.randint(0,100), random.randint(0,100))
        incidents.append({'pos':inc,'time':t,'assigned':None})
    # assign unassigned incidents greedily
    for inc in incidents:
        if inc['assigned'] is None:
```

```
            st = min(stations, key=lambda s: (s[0]-inc['pos'][0])**2 + (s[1]-inc['pos'][1])**2)
            inc['assigned'] = st
            assignments.append((inc['pos'], st, t))
    # draw
    if incidents:
        ax.scatter(*zip(*[x['pos'] for x in incidents]), c='red', label='Incidents')
    ax.scatter(*zip(*stations), c='blue', label='Stations', s=80)
    # draw assignment lines
    for inc in incidents:
        if inc['assigned']:
            ax.plot([inc['pos'][0], inc['assigned'][0]], [inc['pos'][1], inc['assigned'][1]], '--',
color='gray')
    ax.set_xlim(0,100); ax.set_ylim(0,100); ax.set_title(f"Emergency Dispatch t={t}
incidents:{len(incidents)}"); ax.legend()

ani = animation.FuncAnimation(fig, animate, frames=T, interval=300)
plt.close(); HTML(ani.to_jshtml())
```

==========

7>> 7) Pollution Control Monitors — diffusion grid animated heatmap with mitigation

Problem Description

Monitor a city grid for pollution and mitigate hotspots.

Solution Concept

Simulate pollutant diffusion and reduce areas above threshold.

Algorithm Used

Grid Diffusion + Threshold Mitigation

- Pollution spreads via averaging neighbors
- If cell > limit → reduce by mitigation factor

AI Concepts

- **Environment simulation**
- **Distributed sensing**
- **Threshold-based alert systems**

PEAS

- **Performance**: Pollution reduction over time
- **Environment**: 2D grid
- **Actuator**: Mitigation system
- **Sensor**: Pollution sensors

CODE:

```
# Pollution Control Monitors — grid diffusion + mitigation animation
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
```

```
np.random.seed(7)

H,W = 40,40
grid = np.zeros((H,W))
# fixed sources
grid[6,6] += 80
grid[28,32] += 60
frames = []
T = 80

fig, ax = plt.subplots(figsize=(6,6))
def step_diffuse(g):
    # discrete diffusion + decay
    g2 = (np.roll(g,1,0)+np.roll(g,-1,0)+np.roll(g,1,1)+np.roll(g,-1,1))/5 + 0.9*g
    return g2

def animate(t):
    global grid
    grid = step_diffuse(grid)
    # if hotspot above threshold, mitigate (reduce nearby)
    hotspots = np.argwhere(grid > 45)
    for y,x in hotspots:
        grid[max(0,y-1):min(H,y+2), max(0,x-1):min(W,x+2)] *= 0.85
    ax.clear()
    im = ax.imshow(grid, origin='lower', vmin=0, vmax=grid.max()+1)
    ax.set_title(f"Pollution diffusion t={t}")
    ax.axis('off')
    return [im]

ani = animation.FuncAnimation(fig, animate, frames=T, interval=180)
plt.close(); HTML(ani.to_jshtml())
```

==========

8) Streetlight Energy Saver — poles brighten on motion, animated line visualization

Problem Description

Streetlights brighten when motion is detected and dim otherwise to save energy.

Solution Concept

Each pole is an agent with a motion sensor; neighbors partially brighten to form a safe corridor.

Algorithm Used

Event-Driven State Machine
states: BRIGHT ↔ DIM
transition: MotionEvent / Timeout

AI Concepts

- Sensor-based agent
- Event-driven AI

- **Energy optimization**

PEAS

- **Performance: Energy saved**
- **Environment: Street at night**
- **Actuator: Light intensity**
- **Sensor: Motion detectors**

CODE:

```
# Streetlight Energy Saver — animated poles that light on motion
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np, random
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(8)
np.random.seed(8)

n = 40
brightness = np.zeros(n)
T = 300
motion_prob = 0.03

fig, ax = plt.subplots(figsize=(10,2))
```

```
def animate(t):
    ax.clear()
    # random motion events
    events = [random.random() < motion_prob for _ in range(n)]
    for i,e in enumerate(events):
        if e:
            brightness[i] = 1.0
            # neighbors also brighten partially
            if i>0: brightness[i-1] = max(brightness[i-1], 0.7)
            if i<n-1: brightness[i+1] = max(brightness[i+1], 0.7)
    # decay
    brightness[:] = np.maximum(0, brightness - 0.02)
    colors = [(b,b,b) for b in brightness]  # grayscale by brightness
    ax.bar(range(n), brightness, color=colors)
    ax.set_ylim(0,1.05); ax.set_title(f"Streetlight Energy Saver t={t} (brightness sum
{brightness.sum():.1f})")
    ax.axis('off')

ani = animation.FuncAnimation(fig, animate, frames=T, interval=80)
plt.close(); HTML(ani.to_jshtml())
```

==========

9>> 9) Smart Bus Routing — animated buses serving stops, showing wait reduction by higher frequency

Problem Description

Optimize bus frequencies and routes to reduce passenger wait time.

Solution Concept

Simulate passenger arrival + different bus arrival frequencies and plot queue lengths.

Algorithm Used

Queueing Theory + Frequency Optimization

- Avg wait = 1/(2×frequency)

AI Concepts

- **Demand modeling**
- **Queue simulation**
- **Predictive transport planning**

PEAS

- **Performance**: Waiting time
- **Environment**: Bus route
- **Actuator**: Bus dispatch
- **Sensor**: Demand readings

CODE:

```
>> # Smart Bus Routing — simulate buses with two frequencies and show passenger wait visualization
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np, random
```

```
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
random.seed(9); np.random.seed(9)

stops = [(i*10,50) for i in range(8)]
# simulate two scenarios: low-freq and high-freq; show passengers waiting per stop change with time
T = 160
freq_low = 0.4   # probability of bus arriving per time unit
freq_high = 0.95
# passenger arrival prob per stop
arr_prob = 0.12

waits_low = np.zeros((T,len(stops)))
waits_high = np.zeros((T,len(stops)))
q_low = np.zeros(len(stops))
q_high = np.zeros(len(stops))

for t in range(T):
    # arrivals
    arrivals = np.random.rand(len(stops)) < arr_prob
    q_low += arrivals; q_high += arrivals
    # bus arrivals low
    for i in range(len(stops)):
        if random.random() < freq_low:
            q_low[i] = max(0, q_low[i] - random.randint(1,3))
    # bus arrivals high
    for i in range(len(stops)):
        if random.random() < freq_high:
            q_high[i] = max(0, q_high[i] - random.randint(2,5))
    waits_low[t] = q_low
```

```
    waits_high[t] = q_high

fig, ax = plt.subplots(2,1, figsize=(8,6))
def animate(i):
    ax[0].clear(); ax[1].clear()
    ax[0].bar(range(len(stops)), waits_low[i], color='C0'); ax[0].set_title(f'Low Frequency — t={i}')
    ax[1].bar(range(len(stops)), waits_high[i], color='C1'); ax[1].set_title(f'High Frequency —
t={i}')
    for a in ax: a.set_ylim(0, max(waits_low.max(), waits_high.max())+2)
    plt.suptitle('Smart Bus Routing — passenger queues per stop')
ani = animation.FuncAnimation(fig, animate, frames=T, interval=150)
plt.close(); HTML(ani.to_jshtml())
```

==========

10>> 10) Waste Segregation AI — animated classifier demo with simulated features and predicted class color

Problem Description

Classify waste objects into categories for automated sorting.

Solution Concept

Use a small Random Forest ML classifier on synthetic features; show predictions visually.

Algorithm Used

Random Forest Classifier

- **Ensemble of decision trees**
- **High accuracy on small feature sets**

AI Concepts

- **Supervised machine learning**
- **Classification**
- **Computer vision (extendable)**

PEAS

- **Performance: Accuracy, confusion matrix**
- **Environment: Sorting belt**
- **Actuator: Sorting mechanism**
- **Sensor: Feature extractor / camera**

CODE:

```
# Waste Segregation AI — animated demo showing items being classified (synthetic features -> colors)
import matplotlib.pyplot as plt, matplotlib.animation as animation
import numpy as np
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from IPython.display import HTML
plt.rcParams['animation.html'] = 'jshtml'
np.random.seed(10)

# synthetic dataset (5 features)
labels = ['plastic','organic','metal','glass']
X = np.random.rand(400,5)
y = np.random.choice(labels, 400)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.35, random_state=1)
clf = RandomForestClassifier(n_estimators=80, random_state=2)
clf.fit(Xtr, ytr)
probs = clf.predict_proba(Xte)
preds = clf.predict(Xte)

# map classes to colors
color_map = {'plastic':'#1f77b4','organic':'#2ca02c','metal':'#7f7f7f','glass':'#9467bd'}
T = len(Xte)
fig, ax = plt.subplots(figsize=(6,6))

def animate(i):
    ax.clear()
    # show batch of 16 items as squares colored by predicted label, with border true label
    batch = list(range(i, min(i+16, T)))
    for idx, j in enumerate(batch):
        r = idx//4; c = idx%4
        pred = preds[j]; true = yte[j]
        rect = plt.Rectangle((c, 3-r), 0.9, 0.9, color=color_map[pred])
```

```
        ax.add_patch(rect)
        # border indicates correctness
        if pred==true:
            ax.add_patch(plt.Rectangle((c, 3-r), 0.9, 0.9, fill=False, linewidth=2,
edgecolor='white'))
        else:
            ax.add_patch(plt.Rectangle((c, 3-r), 0.9, 0.9, fill=False, linewidth=2,
edgecolor='black'))
        ax.text(c+0.45, 3-r+0.45, pred[:3], ha='center', va='center', color='white', fontsize=8)
    ax.set_xlim(0,4); ax.set_ylim(0,4); ax.axis('off')
    ax.set_title(f'Waste Classifier Demo — items {i+1}..{min(i+16,T)}')
ani = animation.FuncAnimation(fig, animate, frames=range(0, T, 16), interval=900,
repeat=False)
plt.close(); HTML(ani.to_jshtml())

================================================================
```